

NF4 双重量化反量化 CUDA Kernel 实现报告

1. NF4 反量化原理

1.1 NF4 量化简介

NF4 (Normal Float 4-bit) 是由 bitsandbytes 库提出的一种 4-bit 量化格式。其核心思想是：预训练神经网络的权重近似服从正态分布 $N(0, \sigma^2)$ ，因此可以用标准正态分布 $N(0, 1)$ 的 **16 等概率分位点** 作为量化码本 (codebook)，使每个区间内包含的权重数量大致相等，从而最大化信息利用率。这 16 个分位值构成固定的 NF4 查找表：

索引	值	索引	值
0	-1.0000	8	+0.0796
1	-0.6962	9	+0.1609
2	-0.5251	10	+0.2461
3	-0.3949	11	+0.3379
4	-0.2844	12	+0.4407
5	-0.1848	13	+0.5626
6	-0.0911	14	+0.7230
7	0.0000	15	+1.0000

量化时，每个权重被除以其所在块的缩放因子 (absmax) 进行归一化，然后映射到最近的分位值索引 (0-15)，以 4 bit 存储。两个 4-bit 索引打包进一个 `uint8` 字节。

1.2 双重量化

为进一步压缩元数据开销，bitsandbytes 对一级缩放因子 `absmax` 本身再做一次量化，形成**双重量化 (double quantization)** 体系：

- 一级量化**：每 `blocksize` (通常 64) 个元素共享一个缩放因子 `absmax`。经二次量化后，原始的 FP32 `absmax` 被压缩为 `uint8` 索引 `absmax_q`。
- 二级量化**：每 `s2_blocksize` (通常 256) 个一级块组成一个组，共享一个二级缩放因子 `absmax2` (FP16)。同时还有一张 256 项的二级码表 `code2` (FP16[256]) 和一个全局偏移 `offset` (FP32)。

1.3 反量化公式

反量化需要逆向还原上述两级量化过程。对于第 k 个元素：

```

block_idx    = k / blocksize
group_idx    = block_idx / s2_blocksize

absmax_real = code2[ absmax_q[block_idx] ] × absmax2[group_idx] + offset

output[k]    = NF4_TABLE[ nf4_index(k) ] × absmax_real

```

其中 `nf4_index(k)` 是从 `packed_weights` 中解包得到的 4-bit 索引。

2. 基础版 Kernel 实现

基础版 kernel 的核心策略是：**每个线程负责一个 packed byte 的反量化**，产出 2 个输出元素。以下以 `nf4_dequant_kernel_pre.cuh` 中的实现为参考，分步介绍。

2.1 线程映射与 NF4 码表

每个线程处理 1 个 packed byte（含 2 个 4-bit 索引），生成 2 个输出元素。总线程数为 `ceil(n_elements / 2)`，通过全局线程 ID `tid` ——映射到 `packed_weights[tid]`。

16 个 NF4 分位值预存在 `__constant__` memory 中，线程通过索引直接查表：

```
__constant__ float NF4_DEQUANT_TABLE[16] = { -1.0f, -0.6962f, ..., 1.0f };
```

2.2 反量化计算流程

每个线程的完整执行逻辑如下：

Step 1 — 读取与解包：从全局内存读取 1 字节 packed data，分离出高 4 位和低 4 位两个索引，分别对应输出位置 `elem0 = tid * 2` 和 `elem1 = tid * 2 + 1`：

```

uint8_t packed = packed_weights[tid];
uint8_t idx_hi = (packed >> 4) & 0x0F;    // 偶数位索引
uint8_t idx_lo = packed & 0x0F;           // 奇数位索引

```

Step 2 — NF4 查表：用索引查 constant memory 中的码表，获得归一化的浮点值：

```

float val_hi = NF4_DEQUANT_TABLE[idx_hi];
float val_lo = NF4_DEQUANT_TABLE[idx_lo];

```

Step 3 — 还原 absmax（双重量化反解）：通过整数除法确定元素所在的一级块和二级组，再逆向还原真实的缩放因子：

```

int block_idx0 = elem0 / blocksize;
int group_idx0 = block_idx0 / s2_blocksize;
float absmax_real0 = __half2float(code2[absmax_q[block_idx0]])
                    * __half2float(absmax2[group_idx0]) + offset;

```

Step 4 — 块边界复用：相邻的两个元素（偶数位与奇数位）大概率落在同一量化块内。kernel 先为 `elem0` 计算 `absmax_real`，处理 `elem1` 时比较 `block_idx`——若相同则直接复用，仅在跨块时重新计算，避免冗余的全局内存访问和浮点运算：

```
int block_idx1 = elem1 / blocksize;
if (block_idx1 == block_idx0) {
    absmax_real1 = absmax_real0; // 复用
} else { /* 重新计算 */ }
```

Step 5 — Packed Store：将两个 16-bit 输出值（BF16/FP16）的原始位表示打包为一个 `uint32_t`，以单次 32-bit 写入全局内存。这将 2 次 16-bit store 合并为 1 次 32-bit store，减少内存事务数并保证自然对齐：

```
uint16_t bits0 = *reinterpret_cast<uint16_t*>(&out0);
uint16_t bits1 = *reinterpret_cast<uint16_t*>(&out1);
uint32_t packed_out = (uint32_t)bits0 | ((uint32_t)bits1 << 16);
reinterpret_cast<uint32_t*>(output)[tid] = packed_out;
```

2.3 基础版的性能瓶颈

基础版实现功能正确，但存在三个主要瓶颈，将在第 3 节依次优化：

1. **Constant memory 串行化：**NF4 查表时 warp 内线程访问不同索引，触发串行读取；
2. **细粒度全局内存访问：**每线程仅读 1 字节 packed data，远小于 GPU 内存事务粒度（32 字节/sector），带宽利用率低；
3. **整数除法开销：**`block_idx = elem / blocksize` 在 GPU 上延迟高（数十周期），且每线程多次执行。

3. Kernel 优化

在基础版之上，按顺序实施了三项优化。

3.1 优化一：NF4 码表从 `__constant__` 加载到 Shared Memory

问题分析：`__constant__` memory 的特点是当 warp 内所有线程访问同一地址时才能实现广播读取，延迟很低。但 NF4 查表时，warp 内 32 个线程各自查询不同的 4-bit 索引（0-15），访问地址各异，导致 constant memory 的访问被**串行化**，最差情况下需要 16 次串行读取才能满足一个 warp 的请求。

优化方案：在 kernel 启动时，由每个 block 的前 16 个线程协作将 NF4 码表从 `__constant__` memory 加载到 `__shared__` memory：

```
__shared__ float s_nf4_table[16]; // 64 字节
if (threadIdx.x < 16) {
    s_nf4_table[threadIdx.x] = NF4_DEQUANT_TABLE[threadIdx.x];
}
__syncthreads();
```

`__shared__` memory 支持 bank 级别的并行访问：32 个 bank 可以同时响应不同地址的请求。16 个 `float` 映射到不同的 bank，warp 内线程即使访问不同索引也可以在一个周期内完成。

加速效果（实验环境：NVIDIA A100-SXM4-80GB，矩阵 4096×4096，blocksize=64）：V0 中位数 0.1139 ms → V1 中位数 0.0791 ms，加速比 **1.44x**（提升 44%）。这是三项优化中独立效果最显著的一项。

3.2 优化二：向量化读取（每线程读 4 字节 packed_weights）

问题分析：基础版中每个线程仅读取 1 字节 `packed_weights`，产生大量细粒度的全局内存事务。GPU 全局内存事务的最小粒度为 32 字节（一个 sector），1 字节的标量读取会浪费大量带宽。

优化方案：将线程映射从"1 thread = 1 byte = 2 元素"扩展为"**1 thread = 4 bytes = 8 元素**"：

```
// 向量化读取：一次读 4 字节
uint32_t packed4 = reinterpret_cast<const uint32_t*>(packed_weights)[tid_vec];

// 从 packed4 中提取各字节
uint8_t packed_byte = (packed4 >> (b * 8)) & 0xFF;
```

同时写入也从 `uint32_t` (32-bit) 升级为 `uint4` (128-bit = 8 个 FP16/BF16)：

```
reinterpret_cast<uint4*>(out_u32)[tid_vec] =
    make_uint4(out_packed[0], out_packed[1], out_packed[2], out_packed[3]);
```

对于尾部不足 4 字节的边界情况，回退到逐字节标量读取和逐 pack 写入，确保任意矩阵尺寸的正确性。内层循环使用 `#pragma unroll` 展开以减少循环开销。

加速效果：V1 中位数 0.0791 ms → V2 中位数 0.0652 ms，增量加速比 **1.21x**；相对基础版累计加速比达 **1.75x**。值得注意的是，向量化读写单独加到基础版上仅产生 1.09x 加速，但在 shared memory 优化之后叠加效果更强（1.21x），说明两者之间存在正向交互——shared memory 消除了查表瓶颈后，访存带宽成为新的主要瓶颈，向量化此时才能充分发挥作用。

3.3 优化三：用位移代替整数除法

问题分析：基础版中计算 block 索引和 group 索引使用了整数除法：

```
int block_idx = elem / blocksize;        // 整数除法
int group_idx = block_idx / s2_blocksize; // 整数除法
```

GPU 上整数除法指令的吞吐量远低于位移指令。在 NVIDIA GPU 上，32-bit 整数除法的延迟约为数十个周期，而位移仅需 1 个周期。由于每个线程需对 8 个元素执行多次除法运算，累积的延迟开销不容忽视。

优化方案：由于 `blocksize` 和 `s2_blocksize` 总是 2 的幂（如 $64 = 2^6$, $256 = 2^8$ ），在 host 端预计算 $\log_2$ 值，kernel 中用右移替代除法：

```
// Host 端
int log2_bs = log2_pow2(data.blocksize);    // 64 → 6
int log2_s2 = log2_pow2(data.s2_blocksize); // 256 → 8

// kernel 内
int block_idx = elem >> log2_blocksize;     // 右移代替除法
int group_idx = block_idx >> log2_s2_blocksize;
```

`log2_pow2()` 辅助函数通过循环右移计算 2 的幂的对数值。

加速效果： V2 中位数 0.0652 ms → V3 中位数 0.0431 ms，增量加速比 **1.51x**；相对基础版累计加速比达 **2.64x**。位移优化单独加到基础版上仅有 1.03x 提升，但在前两项优化消除了查表和带宽瓶颈后，整数除法的延迟成为关键路径，此时替换为位移带来了显著加速（1.51x），体现了强烈的正向交互效应。

4. 项目架构与实验流程

4.1 项目架构

```
03_nf4_dequant/
├─ run.sh                # 统一流程入口脚本
├─ kernel/
│   ├─ CMakeLists.txt    # CMake 构建系统（自动检测 GPU 架构）
│   ├─ main.cu           # 主程序：文件 IO、kernel 启动、CUDA Events 计时
│   └─ nf4_dequant_kernel.cuh # 优化后的反量化 kernel 实现
│       └─ run_test_ncu.sh # Nsight Compute 性能分析脚本
├─ scripts/
│   ├─ generate_data.py   # 数据生成：用 bitsandbytes 生成 NF4 量化数据 + 参考输出
│   ├─ verify.py         # 正确性验证：CUDA 输出 vs bitsandbytes 参考输出
│   └─ bench_bnb.py      # bitsandbytes 官方库性能基准测试
└─ data/                # 生成的测试数据与输出结果
```

各组件职责：

- `run.sh`：统一入口脚本，支持 `generate`、`build`、`test`、`bench`、`all` 五个子命令，通过命令行选项控制矩阵大小、量化块大小、输出精度等参数。
- `generate_data.py`：使用 `bitsandbytes` 的 `quantize_4bit()` 接口生成 NF4 量化数据，导出为自定义二进制格式，并保存 `bitsandbytes` 的反量化结果作为参考标准。
- `main.cu`：读取二进制文件、分配 GPU 内存、启动 kernel、使用 CUDA Events 精确计时（warmup + repeats 模式，取中位数抗干扰）、输出结果。
- `nf4_dequant_kernel.cuh`：包含全部三项优化的最终 kernel 实现。
- `verify.py`：加载 CUDA 输出与 `bitsandbytes` 参考输出，计算 MAE、MaxError、RMSE、相对 MAE，判定正确性（相对 MAE < 1e-2 为 PASS）。
- `bench_bnb.py`：独立测量 `bitsandbytes` 官方库的反量化性能，支持扫描多种矩阵尺寸，用于对比加速比。

4.2 实验流程

完整的实验流程通过 `./run.sh all` 一键执行，依次完成以下五个步骤：

1. **生成数据**（`generate_data.py`）：调用 `bitsandbytes` 的 `quantize_4bit()` 对随机权重进行 NF4 量化，导出 packed weights、absmax_q、absmax2、code2、offset 等数据为二进制文件（`nf4_weights_*.bin`），同时保存 `bitsandbytes` 的反量化结果作为正确性参考（`nf4_ref_output_*.bin`）。
2. **编译 CUDA kernel**（`cmake + make`）：CMake 自动检测 GPU 架构，编译生成 `nf4_dequant` 可执行文件。
3. **运行 CUDA kernel**（`nf4_dequant`）：读取二进制数据文件，在 GPU 上执行反量化 kernel，输出结果（`cuda_output_*.bin`）和性能数据。
4. **验证正确性**（`verify.py`）：加载 CUDA 输出与 `bitsandbytes` 参考输出，计算 MAE、MaxError、RMSE、相

对 MAE，判定正确性（相对 MAE < 1e-2 为 PASS）。

5. 基准性能对比（`bench_bnb.py`）：独立测量 bitsandbytes 官方库的反量化耗时和带宽，供计算加速比。

性能计时方案：CUDA Events 精确计时，每次 kernel 启动前执行 `cudaDeviceSynchronize()` 确保 GPU 空闲，事件同步后采集单次耗时。收集 `repeats`（默认 100）次数据后排序，报告平均值、中位数、最小值、最大值和基于中位数的有效内存带宽。

5. 实验结果

5.1 正确性验证

矩阵大小	块大小	输出类型	MAE	MaxError	相对 MAE	结果
4096×4096	64	BF16	~2.8e-4	~0.03	~2.6e-5	PASS
4096×4096	64	FP16	~0	~0	~0	PASS
2047×4096	64	FP16	~0	~0	~0	PASS

BF16 的误差来源于 BF16 本身的表示精度（尾数仅 7 bit），与 bitsandbytes 使用 FP32 中间计算再存储为 BF16 存在精度差异；FP16 输出可达到与 bitsandbytes 的 bit-exact 一致。

5.2 性能对比

实验环境：NVIDIA A100-SXM4-80GB，矩阵 4096×4096，blocksize=64，warmup=20，repeats=200。

矩阵大小	块大小	输出类型	CUDA Kernel 中位数耗时	CUDA 带宽	bitsandbytes 中位数耗时	加速比
4096×4096	64	BF16	0.0426 ms	990.98 GB/s	0.0488 ms	1.15x
4096×4096	64	FP16	0.0432 ms	976.31 GB/s	0.0488 ms	1.13x

5.3 各优化阶段加速比

消融实验数据（FP16，矩阵 4096×4096，blocksize=64，repeats=200）：

优化阶段	中位数耗时	有效带宽	相对基础版加速比
基础版 (constant memory + 标量读取 + 整数除法)	0.1139 ms	370.50 GB/s	1.00x
+优化一: Shared Memory 码表	0.0791 ms	533.36 GB/s	1.44x
+优化二: 向量化读取 (4 bytes/thread)	0.0652 ms	647.84 GB/s	1.75x
+优化三: 位移代替除法	0.0431 ms	979.93 GB/s	2.64x

各优化的增量分析与交互效应：

优化	独立加速比	叠加增量加速比	交互效应
Shared Memory NF4 码表	1.44x	1.44x	1.00x
向量化读写	1.09x	1.21x	1.12x
位移代替除法	1.03x	1.51x	1.48x

三项优化叠加后，总加速比为 **2.64x** (0.1139 ms → 0.0431 ms)，有效带宽从 370.50 GB/s 提升至 979.93 GB/s。值得注意的是，向量化和位移优化的独立效果较小 (1.09x、1.03x)，但在前序优化消除了其他瓶颈后，叠加效果显著增强 (1.21x、1.51x)，体现了优化之间的正向交互：shared memory 消除查表瓶颈后，带宽成为新瓶颈，向量化得以发挥；带宽优化后，计算延迟成为新瓶颈，位移替换得以发挥。

6. 国产 GPU 平台适配

`kernel_noncuda/` 目录将优化后的 kernel 移植到三个国产 GPU 平台。三个版本共享相同的算法逻辑与二进制数据格式，可复用 `scripts/verify.py` 进行正确性验证。

平台	目录	编译器	源码后缀	运行时 API 前缀
天数智芯 (Iluvatar)	<code>iluvatar/</code>	<code>clang++</code>	<code>.cu</code>	<code>cuda*</code> (兼容模式)
摩尔线程 (Moore)	<code>moore/</code>	<code>mcc</code>	<code>.mu</code>	<code>musa*</code>
沐曦 (Mutex)	<code>mutex/</code>	<code>mxcc</code>	<code>.maca</code>	<code>mc*</code>

主要适配差异：各平台不直接支持 CUDA 的 `half / __nv_bfloat16` 内建类型，因此改用 `uint16_t` 位操作配合手写的浮点转换函数 (`half_bits_to_float()`、`float_to_half_bits()`、`float_to_bf16_bits()`) 实现等价语义，kernel 模板参数也相应从输出类型改为 `bool OUTPUT_BF16`。除此之外，kernel 核心逻辑 (shared memory 码表、向量化读写、位移索引计算) 与 CUDA 版本保持一致。

各平台目录均提供 `Makefile` 和一键脚本 (`run_*.sh`)，用法与主工程类似。测试数据需在 CUDA 环境预先生成后拷贝至目标机。